Creative Software Design
4. Lab1
Dynamic Memory Allocation

Dept. of Computer Science

Hanyang University

[IMPORTANT] Submission Guide

- Pull to your project directory before each lab session
  - At your VM instance

```
$ cd Projects/{your directory}
$ git pull origin main
$ cd week04_1
```

- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure ***pwd*** at {week}_{session})

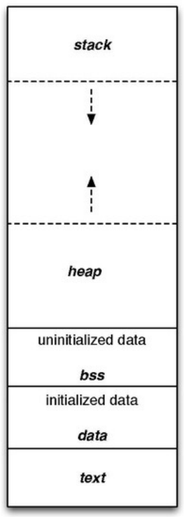
```
# You are at week04_1
$ git add *.cpp
$ git commit –m "submission of week04_1"
$ git push origin main
```

Typical Memory Layout of a Process

- When you run a program, OS allocates memory space for the ***process*** as follows:

(an executing instance of a program)

High



Low

Organized in several segments:

- Stack segment
- Heap segments
- BSS segments (**B**lock **S**tarted by **S**ymbol)
- Data segments
- Text segments

#1 - Linux Memory Layout – 1/2

- Your data will be placed in data, heap, or stack

```
#include <iostream>
#include <iomanip>

using namespace std;

int a = 10, b;

void add(int num1, int num2) {
  int sum = num1 + num2;
  cout << hex << setfill('0') << setw(8);
  cout << "num1 at " << &num1 << endl;
  cout << "num2 at " << &num2 << endl;
  cout << "sum  at " << &sum << endl;
}

int main(void) {
  string txt = "Hello World";
  int* arr = new int[8];
  cout << txt <<  endl;
  b = 20;
  cout << "txt  at " << &txt << endl;
  add(a, b);
  cout << "name at " << ((void*) arr) << endl;
  cout << "b    at " << &b << endl;
  cout << "a    at " << &a << endl;
  cout << "add  at " << (void*) add << endl;
  delete arr;
  return 0;
}
```

ASLR: Changing Memory Layouts

- Linux changes its memory layout at each run
  - Data always are at the same address if the process has static memory layout
  - Attackers easily take private data from the running process
  - For that security issue, Linux changes memory layout for process at every run
- To make the memory layout ***static***

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```


#1 - Linux Memory Layout – 2/2

- Find the section where each data is stored
  - **gdb** also shows the memory map for a process

```
info proc mappings or i proc m
```

```
process 55112
Mapped address spaces:

          Start Addr           End Addr       Size     Offset objfile
      0x555555554000     0x555555555000     0x1000        0x0 /home/wsul/Projects/ITE1015/labs/04-1/mem
      0x555555555000     0x555555556000     0x1000     0x1000 /home/wsul/Projects/ITE1015/labs/04-1/mem
      0x555555556000     0x555555557000     0x1000     0x2000 /home/wsul/Projects/ITE1015/labs/04-1/mem
      0x555555557000     0x555555558000     0x1000     0x2000 /home/wsul/Projects/ITE1015/labs/04-1/mem
      0x555555558000     0x555555559000     0x1000     0x3000 /home/wsul/Projects/ITE1015/labs/04-1/mem
      0x555555559000     0x55555557a000    0x21000        0x0 [heap]
      0x7ffff7a7e000     0x7ffff7a82000     0x4000        0x0
      0x7ffff7a82000     0x7ffff7a8f000     0xd000        0x0 /usr/lib/x86_64-linux-gnu/libm-2.31.so
      0x7ffff7a8f000     0x7ffff7b36000    0xa7000     0xd000 /usr/lib/x86_64-linux-gnu/libm-2.31.so
      0x7ffff7b36000     0x7ffff7bcf000    0x99000    0xb4000 /usr/lib/x86_64-linux-gnu/libm-2.31.so
      0x7ffff7bcf000     0x7ffff7bd0000     0x1000   0x14c000 /usr/lib/x86_64-linux-gnu/libm-2.31.so
      0x7ffff7bd0000     0x7ffff7bd1000     0x1000   0x14d000 /usr/lib/x86_64-linux-gnu/libm-2.31.so
      0x7ffff7bd1000     0x7ffff7bf3000    0x22000        0x0 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7bf3000     0x7ffff7d6b000   0x178000    0x22000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7d6b000     0x7ffff7db9000    0x4e000   0x19a000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7db9000     0x7ffff7dbd000     0x4000   0x1e7000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7dbd000     0x7ffff7dbf000     0x2000   0x1eb000 /usr/lib/x86_64-linux-gnu/libc-2.31.so
      0x7ffff7dbf000     0x7ffff7dc3000     0x4000        0x0
```

Dynamic Memory Allocation in C++

- ***built-in*** operators: **`new`** and **`delete`**
  - No header required
  - **`new`** : creates a variable(instance) of the type(class)
  - **`delete`** : destructs a variable(instance) created by **`new`**
  - **`new`** [] : creates an array of variables(instances) of the type(class)
  - **`delete`**[] : destructs an array created by **`new`**[]

	One instance	Array
Allocate	**`new`**	**`new []`**
Deallocate	**`delete`**	**`delete[]`**

#2 – `malloc/free` vs. `new/delete` - 1

- Investigate each address of pointer p
  - Actually, `new/delete` operator calls `malloc()/free()`

```
#include <iostream>

using namespace std;

int main(void) {
  int i, *p = NULL;
  for (i = 0; i < 10; i++) {
    p = (int*) malloc(sizeof(int));
    cout << p << endl;
    free(p);
  }

  for (i = 0; i < 10; i++) {
    p = new int;
    cout << p << endl;
    delete p;
  }
  return 0;
}
```

#3 – malloc/free vs. new/delete - 2

- But `new` and `delete` serve additional routines
  - `new` calls ***constructor*** and `delete` calls ***destructor***

    ➔ We'll cover constructor/destructor later in detail

```
#include <iostream>

using namespace std;

class Book {
public:
  Book() { cout << "Book()" << endl; }
  ~Book() { cout << "!Book()" << endl; }
};

int main(void) {
  Book* p;
  p = (Book*) malloc(sizeof(Book));
  free(p);
  // p = new Book;
  // delete p;
  return 0;
}
```


#4 – delete vs. delete[] - 1

- `delete` releases data only once even in the array
- But `delete[]` releases every element in the array

```
#include <iostream>

using namespace std;

int main(void) {
  int *p = NULL;

  p = new int[3];
  cout << p << endl;
  delete p;

  p = new int[3];
  cout << p << endl;
  delete[] p;

  return 0;
}
```

#5 – delete vs. delete[] - 2

- `delete` releases data only once even in the array
- But `delete[]` releases every element in the array

```
#include <iostream>

using namespace std;

class Book {
public:
  Book() { cout << "Book()" << endl; }
  ~Book() { cout << "!Book()" << endl; }
};

int main(void) {
  Book *p = new Book[3];
  cout << p << endl;
  delete[] p;

  return 0;
}
```


All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week04_1")

```
# You are at {Your local project directory}/week04_1
$ git add *
$ git commit –m "submission of week04_1"
$ git push origin main
```